

Essentials Of Data Science With R Software - 1: Probability And Statistical Inference
Professor Shalabh
Department of Mathematics & Statistics,
Indian Institute of Technology Kanpur
Lecture No. 05
Calculations with Data Vectors

Hello, friends welcome to the course Essentials of Data Science with R Software 1. In which we are going to handle the topics of probability theory and statistical inference. So, you may recall that in the last couple of lectures, we initiated a brief discussion on the topics of R software, which we are going to use in this course. So, continuing on the same lines in this lecture, we are going to just have a quick review of the mathematical operations related to that data vectors.

One thing you have to keep in mind that in R software whenever we are using the word data vector, this is only the collection of numbers which are separated by the comma written inside the parenthesis using the c command. You should not get confused with the vectors of matrix theory or vector calculus and so on. Because this vector is such a word which is used in different subjects in different ways. So, this R software has a very peculiar advantage and the way R software operates on data vector is pretty different than what other software's do.

So, that is why it is very important for us to understand how the R is working with the data vectors why, because, later on you will see that whenever we are trying to insert any data or whenever we are trying to input any data, in most of the cases we are going to use that data vector. So, it is important for you to understand that when you are trying to Add, Multiply, Divide, etc. type of operations on data vectors, then what R is doing inside the computer? What the R is doing behind the screen? Once you understand that thing that will help you in making the statistical calculation faster and easier.

So, in this lecture, we are just going to handle the simple basic operations of mathematics over the data vectors. So, let us begin our lecture. So, now, you can see here.

(Refer Slide Time: 02:30)

Calculations with Data Vectors

How R behaves with data vectors?

What happens when a scalar is added/subtracted/multiplied/divided in a data vector?

That my basic objective is this, how R behaves with the data vectors and what really happens to R or how the R operates when a scalar or a data vector is operated on a data vector. Scalar versus scalar that we already have done in the last lecture.

(Refer Slide Time: 02:49)

Addition in Data Vectors: $x + y$

Data vector

Scalar

```
> c(3,4,5,6) + 5
```

[1] 8 9 10 11

3+5, 4+5, 5+5, 6+5

R Console

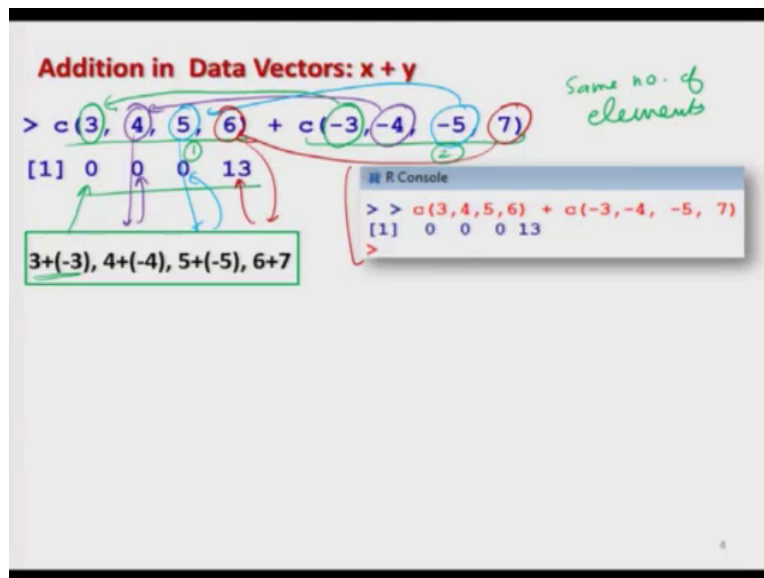
```
> c(3,4,5,6) + 5
```

[1] 8 9 10 11

So, now, let me take a simple example, and then I will try to show you it on the R console also. Suppose, if I take here a data vector which is consisting of four numbers 3,4,5,6, and we try to add here a scalar. So, you can see here, this is a scalar, and this here is a data vector. So, now, in case if you try to add it, then you can see here what is happening, this 5 is being added to each of the elements inside the data vector. So, what is happening $3 + 5$, $4 + 5$, $5 + 5$, $6 + 5$, which is coming from here.

And then finally, you get an outcome here, 8, 9,10,11, $3 + 5$ is 8, $4 + 5$ is 9, $5 + 5$ is 10, and $6 + 5$ is 11. And here is the screenshot, I will try to show you and I try to do it on the R console, but before that let us try to understand these operations.

(Refer Slide Time: 03:44)



Now, in case if you are trying to use this addition operator on two data vectors, then what happens? So, you can see here I am trying to take here the data vector number here 1 and data vector number here 2. Now, once you try to add these two data vectors and what you have to notice that both these data vectors have the same number of elements, the data vector 1 has 1,2,3,4 elements and data vector 2 has 1,2,3,4 elements.

So, you have to just keep this thing in mind because, anyway I am going to show you that when these numbers are not the same, then what happens and how R operates, but here if you try to see what is happening? Your outcome is coming out to be here 0,0,0, and 13. What does R is trying to do? If you try to understand it, the simple rule is R is doing the addition, element-wise. Element wise means 3 is the first element of data vector 1 and -3 is the first element of data vector 2, so, both are going to be added. So, this becomes here $3 + -3$, which is equal to 0.

And similarly, once you try to take the second element of data vector 1 and the second element of data vector 2, which are 4 and -4, then these two are getting added, and it gives you here $4 + -4$, which is equal to here 0. And similarly, if you try to take the third element of data vector 1 and the third element of data vector 2, which are 5 and -5, respectively. Then they are going to be added $5 + -5$, which is here, 0, and 0 is coming over here.

And similarly, the last value, which is the fourth element of the data vector 1 and fourth element of the data vector 2, they are getting added $6 + 7$, and then you are getting here the answer 13. And you can see here, this is the screenshot of the same operation when you try to do it on the R console. So, what you have to observe here that this R is adding element-wise.

(Refer Slide Time: 06:05)

Addition in Data Vectors: $x + y$

```
> c(3,4,5,6) + c(7,8)
[1] 10 12 12 14
```

3+7, 4+8, 5+7, 6+8

R Console

```
> c(3,4,5,6) + c(7,8)
[1] 10 12 12 14
>
```

Addition in Data Vectors: $x + y$

```
> c(3,4,5,6) + c(7,8)
[1] 10 12 12 14
```

3+7, 4+8, 5+7, 6+8

R Console

```
> c(3,4,5,6) + c(7,8)
[1] 10 12 12 14
>
```

Now, what will happen in case if the number of elements in the two data vectors are not the same. So, suppose I try to take care of data vector say number 1, which has got here 4 elements 3,4,5,6. And the second data vector, which has got here the elements, 7 and 8, just two elements. And remember one thing here, I am trying to do that the number of elements in the data vector 2 here, they are the multiple of or the exact multiple of the total number of

elements in the first data vector. So, let me try to clear the screen and so that I can show you the operations more clearly.

Now, if you try to look at the outcome, what is happening your outcome here is 10,12,12,14. So now, if you try to see what is happening, this whole data vectors, 7 and 8, that is the data vector 2. This comes over the first data vector, and it operates on the first 2 elements of the data vector 1 and this gives you here $3 + 7$ and $4 + 8$. And this gives you an answer $3 + 7$ is 10, and $4 + 8$ is 12. So, these two elements now, you can understand very easily. Now, what it does? That once again, it takes the same data vectors 7 and 8 and it tries to operate on the remaining 2 elements of the data vector 1 which are 5 and 6.

So, finally, it gives you here $5 + 7$ and $6 + 8$, which are giving you, in turn, the answer 12 and here 14. So, you can see here that these data operations are getting distributed with respect to the numbers. So, once you have done the number of data elements in that second vector, which are an exact multiple of the number of data values in the first data vector, the smaller number of data vectors is getting repeated, again and again, that is what is happening. So, this is a very particular and peculiar characteristic of the R software.

(Refer Slide Time: 08:03)

Addition in Data Vectors: $x + y$

```
> c(3,4,5,6) + c(7,8,9) # Warning message
[1] 10 12 12 13
Warning message:
In: c(3, 4, 5, 6) + c(7, 8, 9)
longer object length is not a multiple of
shorter object length
```

Handwritten calculations and annotations:

- Green box around output: $10, 12, 12, 13$
- Green box around calculation steps: $3+7, 4+8, 5+9, 6+7$
- Blue box around warning message: "Warning message: In: c(3, 4, 5, 6) + c(7, 8, 9) longer object length is not a multiple of shorter object length"
- Blue box around handwritten calculation: $6+7=13$ with the word "Confused" written below it.

Now, an obvious case comes. Now, I am saying, again and again, that here the number of elements in the smaller data vector, they are the exact multiple of the total number of elements in the first data vector. Now, suppose it does not hold true and the total number of elements in the smaller data vector, they are not the exact multiple of the total number of

elements in the first data vector, then what will happen? So, let us try to see this thing, as an example.

So, if you try to see here in the data vector 1, you have four elements here 3,4,5,6, and data vector here 2, you have here three elements 7,8,9. So, what will happen here, that the same concept will be operated and this data vector, the values 7,8,9, they will come to the first data vector and they will try to operate element-wise. So, what will happen the first element of the data vector 1, which is here, 3, that will be added with the first element of the data vector 7, and we get here $3 + 7$, and this is giving you answer 10. Now, the same operation will happen with the remaining two elements of that data vector 1, which are 4 and 5, with the remaining values in that data vector 2, which are 8 and 9. So, we get here, $4 + 8$ and $5 + 9$.

Now, as you can see in the earlier slide, what happened if you try to see here, that in the second time, the remaining this element was coming to the first data vector once again. So now the same operation will happen again, that entire data vector will come here, but now you can see here that hereafter 6, there are no values. So now, are we getting confused? What will happen here? That there should have been here 3 value like 6 something here and then now this 7,8,9, is coming over here. So, now what it will try to do? It will try to add 6 and 7 and it will give us the answer 13. But after that, what to add in the first values that R is not getting.

So, now, after this, the R gets confused here, what to do? And R is not actually sure whether you have made this data vector intentionally or this is by some mistakes. So, that is why it will add 6 and 7 and it will give you here an answer 13. But it will also give you a warning message, warning message is trying to say that the longer object length is not a multiple of the shorter object length that means, R is trying to help you that please try to have a look and please verify if you have not made any mistake and this is your plan that you are trying to give the total number of elements in the two data vectors which are not the exact multiple.

So, now, you can see here that I have taken the example of this addition and I have given you the methodology or the basic concept that how R works with that data vector. So, there are three situations when the total number of elements in that two data vectors are the same. And the second condition is when the total number of elements in the two data vectors are not the same. And under the second case also, there are two options that the total number of elements in the data vector with the smaller number of elements is an exact multiple of the total

number of elements in the other data vector or the second option is it is not an exact multiple. So, under these conditions, I have shown you that how R works.

Now, the same concept is going to continue to work when we are trying to do the mathematical operations like subtraction, division, power operators, multiplication. So, now what I will try to do? I have explained to you that how R is working with the data vectors in the addition. Now, I will try to take the examples of subtraction, multiplication, division, and power operation. And I will try to show you how it works. But the basic concept and the way, the approach of working of R on the data vectors will remain the same.

(Refer Slide Time: 12:35)

Addition in Data Vectors: $x + y$

```
R Console
> c(3,4,5,6) + c(7,8,9)
[1] 10 12 14 13
Warning message:
In c(3, 4, 5, 6) + c(7, 8, 9) :
  longer object length is not a multiple of shorter object length
>
```

Subtraction in Data Vectors: $x - y$

```
> c(3,4,5,6) - 5
[1] -2 -1 0 1
```

Handwritten annotations on the subtraction slide:

- data vector* (circled in blue) pointing to `c(3,4,5,6)`
- Scalar* (written in blue) pointing to `- 5`
- Arrows pointing from the result `[1] -2 -1 0 1` to a box containing `3-5, 4-5, 5-5, 6-5`

And, this is here the screenshot of the same operation. Now, let me try to take here the example of subtraction. And you can see here that I have taken here our data vector like this.

And here is a scalar. So now, what is happening that is 5 is entering into the data vector and it is trying to operate the subtraction sign on each of the elements. So, this becomes $3 - 5$, $4 - 5$, $5 - 5$, $6 - 5$ just like addition, and it gives you the answer $3 - 5$ is equal to -2 , $4 - 5$ is -1 , $5 - 5$ is 0 and $6 - 5$ is 1 . So, this is how this R is working.

(Refer Slide Time: 13:16)

Subtraction in Data Vectors: $x - y$

```
> c(3, 4, 5, 6) - c(-3, -4, -5, 7)
[1] 6 8 10 -1
```

$3 - (-3)$, $4 - (-4)$, $5 - (-5)$, $6 - 7$

R Console Screenshot:

```
> c(3, 4, 5, 6) - c(-3, -4, -5, 7)
[1] 6 8 10 -1
> |
```

And I will try to show you on the R console also. And now, just like the case of an addition, if you try to take two data vectors, which have got the same number of elements, then the operation of subtraction is again going to be the element-wise that 3 and -3 will be subtracted or I will say this that the element-wise operation will happen on data vector 1 and data vector 2 and the first element of data vector one will be subtracted by the first element in the data vector 2 and the same process will continue for the respective numbers at a respective position inside the two data vectors.

So, the same operation comes over here. So, you can see here 3 minus, -3 , because it becomes here $3 + 3$, which is 6 , then the second element or the elements at the second portion or in that two data vector that will become here, 4 and here -4 . So, this becomes 4 minus, -4 , which is here 8 . And similarly, you get here the remaining 2 elements and this is here the screenshot. So, you can see here it is not difficult at all.

(Refer Slide Time: 14:19)

Subtraction in Data Vectors: $x - y$

```
> c(3,4,5,6) - c(7,8)
[1] -4 -4 -2 -2
```

3-7, 4-8, 5-7, 6-8



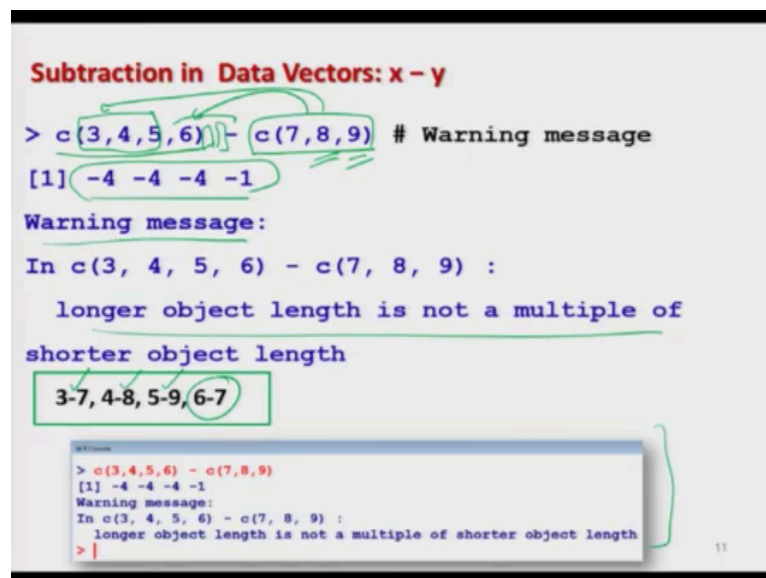
```
R Console
> c(3,4,5,6) - c(7,8)
[1] -4 -4 -2 -2
> |
```

10

And the same thing will happen if you try to operate the two data vector which are not of the exact length. But in the first case, there are two data vectors. The data vectors here number 1 which has 4 elements 3,4,5, and 6. And the second data vector has two elements only 7 and 8 and the second data vector which is 2 that is an exact multiple of the total number of elements in the data vector 1 which is 4. So, now the same thing will happen that this 7 and 8. That will be transported to the data vector 1 and it will be operated on the first two elements and then it will be transported again to the data vector 1 and it will be operated on the remaining elements of the data vector 1.

So, now this will become here simply, 3 -7, 4 -8. And then once again, 5 -7, then 6 -8, and that is going to give you the answer -4, -4, -2 -2. So, you can see here that these operations are pretty straightforward. And the basic concept and the basic methodology is the same that happened in the case of addition also.

(Refer Slide Time: 15:28)



```
Subtraction in Data Vectors: x - y
> c(3,4,5,6) - c(7,8,9) # Warning message
[1] -4 -4 -4 -1
Warning message:
In c(3, 4, 5, 6) - c(7, 8, 9) :
  longer object length is not a multiple of
shorter object length
3-7, 4-8, 5-9, 6-7
```

The screenshot shows an R console window. At the top, the title 'Subtraction in Data Vectors: x - y' is written in red. Below it, the command `> c(3,4,5,6) - c(7,8,9)` is entered, followed by a comment `# Warning message`. The output is `[1] -4 -4 -4 -1`. Below the output, a warning message is displayed: 'Warning message: In c(3, 4, 5, 6) - c(7, 8, 9) : longer object length is not a multiple of shorter object length'. Handwritten green circles and arrows highlight the vectors `c(3,4,5,6)` and `c(7,8,9)`, the warning message, and the calculation `3-7, 4-8, 5-9, 6-7` which is circled in green. A smaller inset window at the bottom shows the same console output.

And similarly, if you try to take here the example, where the total number of elements in the two data vectors are not the exact multiple, then what happen, the same thing will happen, what happened in the case of an addition, that you will get an outcome in which as far as possible, the element-wise subtraction will happen. And after that, whatever is left that will be operated, but a warning message will come. So, you can see here, that in that data vector 1 we have here for element 3,4,5,6. And in the second data vector, we have here three elements 7,8,9.

So, these 7,8,9, from the data vector 2, will be transported to the first three elements of the data vector 1, 3,4,5, and the element-wise subtraction will happen. That is 3 -7, 4 -8, 5 -9. Now after this 6 is left, so there should have been two more values. That is what R is thinking, and it tried to transport this 7,8,9, to this position, here, 6. So, now the remaining element 6 will be subtracted by the first element of the second data vector that is 6 -7, and it will give you the answer, -1. But what to do with 8 and 9, that R does not know.

Because we have not supplied the numbers who are there. So, it will not do anything, but certainly, it will give it give a warning message that the longer object length is not a multiple of the shorter object length. So, you can see here that this is the operation. And this is the outcome of this operation. And this is here, the screenshot. So, this is how this subtraction will also work. Now, before going to another operation, when we try to show you these things on the R console.

(Refer Slide Time: 17:13)

```
> c(3,4,5,6) + 2
[1] 5 6 7 8
> c(3,4,5,6) - 2
[1] 1 2 3 4
>
> c(3,4,5,6) + c(1,2)
[1] 4 6 6 8
> c(3,4,5,6) - c(1,2)
[1] 2 2 4 4
>
> c(3,4,5,6) + c(1,2,3,4)
[1] 4 6 8 10
> c(3,4,5,6) - c(1,2,3,4)
[1] 2 2 2 2
>
> c(3,4,5,6) + c(1,2,3)
[1] 4 6 8 7
Warning message:
In c(3, 4, 5, 6) + c(1, 2, 3) :
  longer object length is not a multiple of shorter object length
> c(3,4,5,6) | c(1,2,3)
```

```
[1] 1 2 3 4
>
> c(3,4,5,6) + c(1,2)
[1] 4 6 6 8
> c(3,4,5,6) - c(1,2)
[1] 2 2 4 4
>
> c(3,4,5,6) + c(1,2,3,4)
[1] 4 6 8 10
> c(3,4,5,6) - c(1,2,3,4)
[1] 2 2 2 2
>
> c(3,4,5,6) + c(1,2,3)
[1] 4 6 8 7
Warning message:
In c(3, 4, 5, 6) + c(1, 2, 3) :
  longer object length is not a multiple of shorter object length
> c(3,4,5,6) - c(1,2,3)
[1] 2 2 2 5
Warning message:
In c(3, 4, 5, 6) - c(1, 2, 3) :
  longer object length is not a multiple of shorter object length
> |
```

So, that you get confident that yes, these things are happening. So, if you try to see here that I try to take care of the four elements, 3,4,5, and see here 6. And I try to, suppose add here, number 2. You can see here that this number 2 has been added on each of the elements in 3,4,5, and 6, and the answer comes out to be 5,6,7,8. And similarly, if I try to subtract it, then what will happen, this number 2 is going to be subtracted from each of the elements in the data vector. So, this will become 3 -2, 4 -2, 5 -2, and 6 -2, and the answer will come out to be 1,2,3,4. There is no issue at all.

Now, similarly, if I try to take here the same example, but I try to take the first data vector at c 3,4,5,6, and the second data vector here, say 1 and 2. So you can see here, what we expect that this, 1, 2 will be added first to the 3, 4, and then to the 5, 6. So, if you try to enter here,

you get this outcome, $3 + 1$, which is 4, $4 + 2$, which is 6, $5 + 1$, this is 6, and $6 + 2$, which is 8. And similarly, if you try to do here, the subtraction in the same data vector, you get the same operation, that is $3 - 1$, which is here, 2, $4 - 2$ is here, 2, $5 - 1$ here is 4, and $6 - 2$ here is 4.

Now, in the same operation, if you try to take here the equal number of elements in both the data vectors, then what happened that in the second data vector. Now, I try to take here four numbers 1,2,3,4. So the total number of elements in the first 10-second data vector, they are the same. So, you can see here, once you try to add them the elements in the second data vector 1,2,3,4, they are going to be added element-wise in the first data vector 3,4,5,6. And you get an outcome here, 4,6,8, and 10.

And similarly, if you try to do here a subtraction with the same data vector, the four elements of this 1,2,3,4, they are going to be subtracted from the first 4 elements of the data, vector 1 and you get here and answer, $3 - 1$ is 2, $4 - 2$ is 2, $5 - 3$ is 2 and $6 - 4$ is 2. So, this is how you can get these operations. Now, I try to do here one thing I try to make the second data vector to have only the 3 elements whereas the first data vector, it has got four elements.

So now if you try to add them here, you will get here the answer with a warning that these three numbers 1,2,3, they are going to be added on the first three elements of the data vector 1,3,4,5. But for the 6th, there is only one element, 1 which is going to be added, and these two elements 2 and 3 the first data vector have no place to add them. So, that is why it is trying to give you here a warning message.

And similarly, if you try to do here the subtraction also, the same thing will happen here, that if you try to see that, this 1,2,3 that is going to be subtracted from the first three elements 3,4,5, and then 6 is going to be subtracted by 1 and so, you get here an answer triple 2 and 5. And now, after this, these two numbers 2 and 3 do not find any place in the first period of the data vector. We add the subtraction can be done. So, it is not doing anything, but it is simply giving you a warning message. So, you can see here that these operations are not difficult at all.

(Refer Slide Time: 20:45)

The slide is titled "Multiplication in Data Vectors: $x * y$ ". It features handwritten annotations in green ink. The R code `> c(4,5,6,8) * 3` is shown, with "data vector" written above `c(4,5,6,8)` and "Scalar" written above `* 3`. The console output is `[1] 12 15 18 24`. Below the code, a box contains the calculations `4 x 3, 5 x 3, 6 x 3, 8 x 3`, which is circled in green. To the right, a screenshot of the R Console shows the same code and output. A green bracket on the right side of the slide groups the code, output, and the calculation box.

And now, let us try to consider the multiplication. So, now, I try to take similar examples, and once again a similar type of data operations will be there. The basic fundamentals are going to be the same and the way the R is going to operate that is also going to be the same. So, now, I try to take here first data vector which has got here 4 elements say 4,5,6,8, and I try to multiply it by the number 3 which is a scalar.

So, now, we have a data vector and a scalar. So, now, a similar operation will happen that the 3 is going to be multiplied on each of this element inside the data vector 4 into 3, 5 into 3, 6 into 3, and 8 into 3. So, the answer will come out to be 12 15 18 24. So, the same operation which is here indicated here that will happen. This means, the scalar is going to be multiplied inside the data vector on every element of the data vector and this is here the screenshot of the same thing.

(Refer Slide Time: 21:45)

Multiplication in Data Vectors: $x * y$

```
> c(3,4,5,6) * c(-3,-4,-5,7)
```

[1] -9 -16 -25 42

3 x (-3), 4 x (-4), 5 x (-5), 6 x 7

R Console

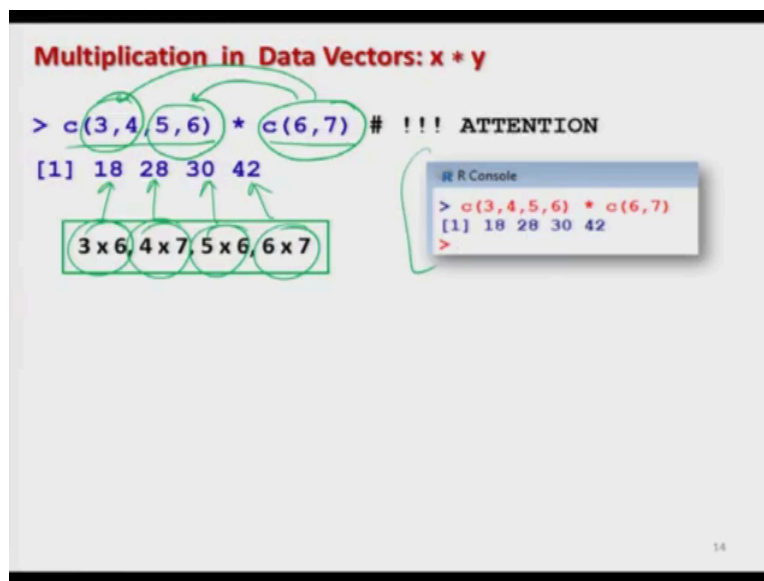
```
> c(3,4,5,6) * c(-3,-4,-5,7)
[1] -9 -16 -25 42
>
```

13

Now, similarly, if you try to take another example, where you are trying to multiply 2 data vectors and suppose the total number of elements in the two data vectors, they are the same. So, you can see here this is your data vector 1 and this is your data vector 2 and both the data vectors have got the same number of elements that is 4. So, now, once again just like we have learned that in the case of addition and subtraction, this operation is going to be element-wise that means, the first element of the data vector 1 is going to be multiplied by the first element of the data vector 2, the second element of data vector 1 is going to be multiplied by the second element of data vector 2.

And similarly, this process will continue. So, now, what will happen here, this -3 will be multiplied by 3, -4 will be multiplied by 4, -5 will be multiplied by 5, and 7 will be multiplied by 6. So, the answer will come out to be 3 into -3 is -9, then 4 into -4 is -16, 5 into -5 is -25, and 6 into 7 is 42. So, you can see here this operation is being done and this is the screenshot which is the outcome of this operation on the R console.

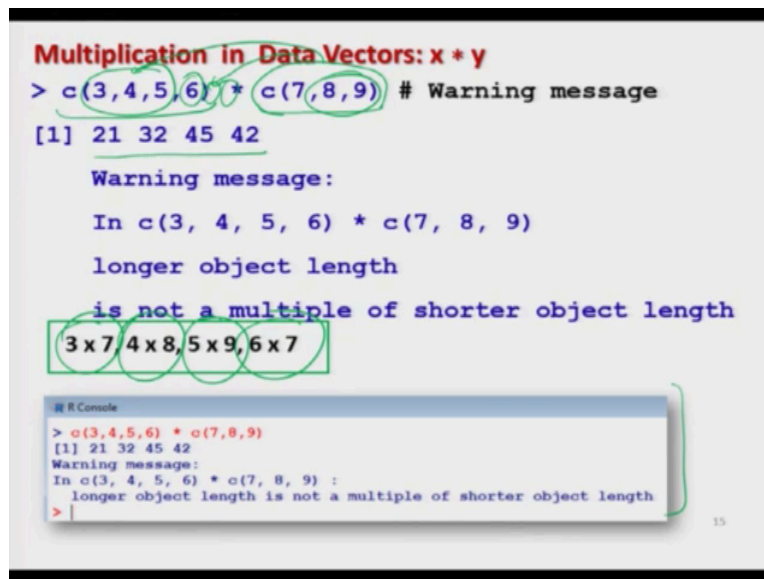
(Refer Slide Time: 22:59)



Now, similarly, if you try to take here the two data vectors, where the first data vector has got 4 elements and the second data vector has got only two elements. But the condition is that number of elements in the second data vector, they are smaller than the number of elements in the first data vector, but they are the exact multiples. So, now, the same thing will happen the 6 and 7 numbers in this data vector 2, they will be transported to the first data vector and they will be multiplied by 3 and 4, which are the first 2 positions in the data vector 1.

Now, once again, this 6 and 7 will be transported to data vector 2, and these two numbers will be multiplied by the next 2 positions after the second element. So, what will happen here, this will become here 3 into 6, then 4 into 7, then 5 into 6, and then 6 into 7, and this will give you an answer, say 18, 28, 30, and 42. That is pretty simple, you know the multiplication, and this is here, the screenshot of the same operation when you try to do it on the R console.

(Refer Slide Time: 24:04)



The screenshot shows an R console session. At the top, the title "Multiplication in Data Vectors: x * y" is displayed in red. Below it, the command `> c(3,4,5,6) * c(7,8,9) # Warning message` is entered. The output is `[1] 21 32 45 42`. A warning message follows: "Warning message: In c(3, 4, 5, 6) * c(7, 8, 9) longer object length is not a multiple of shorter object length". Below the warning, the calculation `3 x 7, 4 x 8, 5 x 9, 6 x 7` is shown in a box. At the bottom, a smaller screenshot of the R console is shown, displaying the same command and output, with the warning message highlighted.

```
Multiplication in Data Vectors: x * y
> c(3,4,5,6) * c(7,8,9) # Warning message
[1] 21 32 45 42

Warning message:
In c(3, 4, 5, 6) * c(7, 8, 9)
longer object length
is not a multiple of shorter object length
3 x 7, 4 x 8, 5 x 9, 6 x 7

R Console
> c(3,4,5,6) * c(7,8,9)
[1] 21 32 45 42
Warning message:
In c(3, 4, 5, 6) * c(7, 8, 9) :
longer object length is not a multiple of shorter object length
>
```

Now, the same operation that if you try to take here two data vectors, in which the first data vector has got here, four elements 3,4,5,6, and the second data vector has got only 3 elements 7,8,9. So, the total number of elements in the second data vector, they are not the exact multiple of the total number of elements in the first data vector. So, in this case, a similar operation will happen.

Now you know that this 7,8,9, will be transported to the first three elements of the data vector 1 and we will have 3 into 7, 4 into 8, 5 into 9. Now, this data vector 7,8,9, will be transported to the first data vector and the first element is here 6. So, 6 and 7 are going to be multiply. But what to multiply with 8 and 9 that is not clear to R. So, it will not do anything but it will simply give you a warning message and it will give you the output 21 30 45 40. And this is here, the screenshot of the same operation when you try to do it on the R console. So, you can see here it is not a very difficult thing.

(Refer Slide Time: 25:07)

Division in Data Vectors: x / y

data vector
scalar

```
> c(2,4,6,8) / 2  
[1] 1 2 3 4
```

$2/2, 4/2, 6/2, 8/2$


```
> c(2,4,6,8) / 4  
[1] 0.5 1.0 1.5 2.0
```

$2/4, 4/4, 6/4, 8/4$

R Console

```
> c(2,4,6,8) / 2  
[1] 1 2 3 4  
> c(2,4,6,8) / 4  
[1] 0.5 1.0 1.5 2.0  
> |
```

And similarly, the division will also happen. That if you try to take the first case where you are trying to divide the data vector, and then a scalar. So here the data vector has got here, four values 2,4,6, and 8, and scalar here is 2. So now, the same operation will happen here, once again, that this division operator will be operated on each of the elements inside the data vector 2,4,6, and 8. So, now you can see here, this will become a 2 by 2, 4 by 2, 6 by 2, and 8 by 2. And the answer will come out to be 1,2,3,4. Pretty straightforward and simple thing for you.

And similarly, if you try to divide the same data vector by 4, then the answer will be coming out to be in the fraction. So, I just wanted to show you that, that it is not really true that the answer is always in the integer. But that can be any real number, any fraction value also. So, you can see here, in this case, each of the elements in the first data vector 2,4,6,8, that is going to be divided by 4 and the answer comes out to be 2 divided by 4, 4 divided by 4, 6 divided by 4, and 8 divided by 4 and this answer comes out to be 0.5, 1.0, 1.5, and 2.0. And this is here the screenshot of both these operations. So, we can see here it is pretty simple, straightforward, easy.

(Refer Slide Time: 26:33)

Division in Data Vectors: x / y

```
> c(4, 8, 12, 16) / c(-2, -4, -3, 8)
```

[1] -2 -2 -4 2

$4 / (-2), 8 / (-4), 12 / (-3), 16 / 8$

R Console

```
> c(4, 8, 12, 16) / c(-2, -4, -3, 8)
[1] -2 -2 -4 2
> |
```

And now, in case if you try to divide the two data vectors having the same number of elements, then once again the similar thing will be executed. For example, if I try to take here the first data vector to have the numbers 4, 8, 12, 16. So, there are 4 values and in the second data vector I have once again four values, but they are different -2, -4, -3, and 8. So, now, what will happen once you try to divide it then once again the operation is going to be element-wise that the first element of the data vector 1 is going to be divided by the first element of the data vector 2, the second element of data vector 1 is going to be divided by the second element of the data vector 2 and so, on.

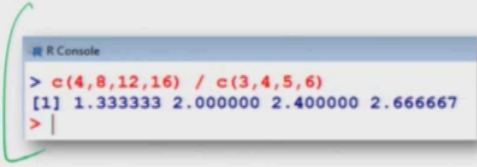
So, what will happen this 4 which is the first element of data vector 1, and will be divided by the first data element of the second data vector which is -2. So, we will be operating 4 divided by -2, which is equal to here -2. And similarly, this 8 will be divided by -4, 12 will be divided by here -3, and 16 will be divided by here 8 just element-wise. So, this will happen and you will get an answer here at -2, -2, -4, and 2 that you know the division, it is not difficult for you. So, and this is here the screenshot if you try to do it on the R console, you will get the same thing.

(Refer Slide Time: 27:54)

Division in Data Vectors: x / y

```
> c(4,8,12,16) / c(3,4,5,6)
[1] 1.333333 2.000000 2.400000 2.666667
```

$4 / 3, 8 / 4, 12 / 5, 16 / 6$



R Console

```
> c(4,8,12,16) / c(3,4,5,6)
[1] 1.333333 2.000000 2.400000 2.666667
>
```

18

Now, in case if you try to take here the numbers in the two data vectors, so, you can see here in this example, that the answer can be in real numbers in fraction also. So, now, I try to take your 2 data vectors here. And first, the data vector has some element 4, 8, 12, 16, and the second data vector has got the element 3, 4, 5, 6. So, once you are trying to divide it, this will give you an answer 4 divided by 3, 8 divided by 4, 12 divided by 5, and 16 divided by 6. So, this is here like this and you get here an answer like this one and this is a screenshot. So, I just wanted to show you that it is not true that the answers are always an integer.

(Refer Slide Time: 28:41)

Division in Data Vectors: x / y

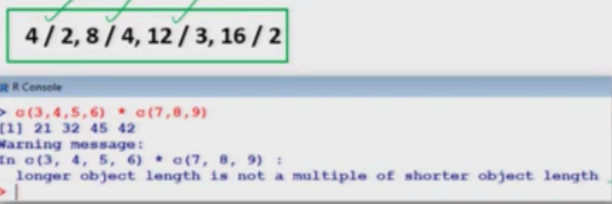
```
> c(4,8,12,16) / c(2,4,3) # Warning message
[1] 2 2 4 8
```

Warning message:

In $c(4, 8, 12, 16)/c(2, 4, 3)$:

longer object length is not a multiple of shorter object length

$4 / 2, 8 / 4, 12 / 3, 16 / 2$



R Console

```
> c(3,4,5,6) * c(7,8,9)
[1] 21 32 45 54
Warning message:
In c(3, 4, 5, 6) * c(7, 8, 9) :
longer object length is not a multiple of shorter object length
>
```

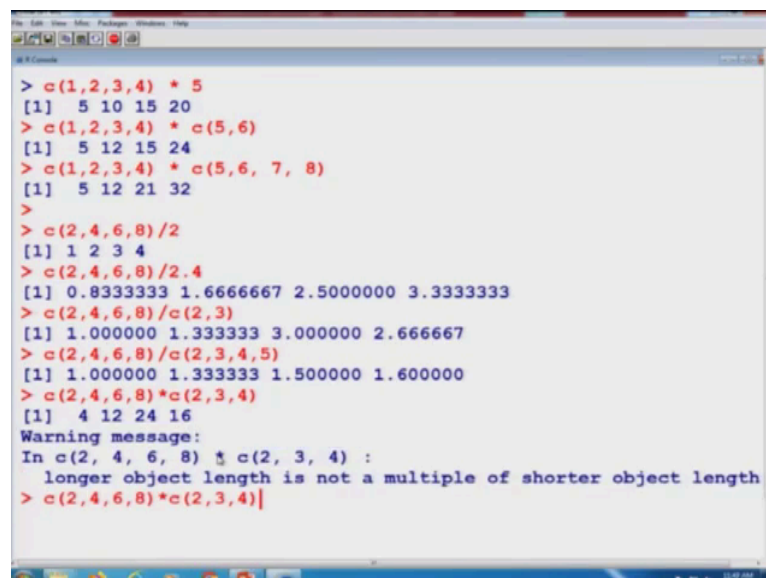
19

And now, similarly, if you try to take here another division in which the total number of elements are not going to be an exact multiple of the total number of elements in the two data

vectors, then once again the similar operation will happen, and you will get a warning in the outcome. So, for example, if I try to take here, four elements in the data vector 1, which are going to be divided by the 3 elements in the data vector 2. So, what will happen the three elements in the data vector 2 which are a 2,4,3, they will be transported to the first three positions of the data vector 1. And 4, 8, and 12, which are the 3 numbers on the first, second, and third positions in the data vector 1 they will be divided by these numbers respectively.

So, what will happen here, you will get here 4 divided by 2, 8 divided by 4, and 12 divided by 3. Now, after this, it will be transported once again maybe because 1 number is left in the data vector 1. So, this 16 will be divided by 2, but after this, there are no elements where this 4 and 3 can be divided. So, R will get confused, R will not come to know what to do. So, but since there is no number, so, it will give you the warning and it will try to indicate. Yes, please try to see that after 16 there are no numbers. So, I am unable to do any operation, but please check this is what exactly you want it to do. And then you will get an answer here 16 by 2, which is here 8. So, and this is here with a screenshot of the same operation.

(Refer Slide Time: 30:23)



```

> c(1,2,3,4) * 5
[1] 5 10 15 20
> c(1,2,3,4) * c(5,6)
[1] 5 12 15 24
> c(1,2,3,4) * c(5,6, 7, 8)
[1] 5 12 21 32
>
> c(2,4,6,8)/2
[1] 1 2 3 4
> c(2,4,6,8)/2.4
[1] 0.8333333 1.6666667 2.5000000 3.3333333
> c(2,4,6,8)/c(2,3)
[1] 1.000000 1.333333 3.000000 2.666667
> c(2,4,6,8)/c(2,3,4,5)
[1] 1.000000 1.333333 1.500000 1.600000
> c(2,4,6,8)*c(2,3,4)
[1] 4 12 24 16
Warning message:
In c(2, 4, 6, 8) %*% c(2, 3, 4) :
  longer object length is not a multiple of shorter object length
> c(2,4,6,8)*c(2,3,4)|

```

```

[1] 5 12 15 24
> c(1,2,3,4) * c(5,6, 7, 8)
[1] 5 12 21 32
>
> c(2,4,6,8)/2
[1] 1 2 3 4
> c(2,4,6,8)/2.4
[1] 0.8333333 1.6666667 2.5000000 3.3333333
> c(2,4,6,8)/c(2,3)
[1] 1.0000000 1.3333333 3.0000000 2.6666667
> c(2,4,6,8)/c(2,3,4,5)
[1] 1.0000000 1.3333333 1.5000000 1.6000000
> c(2,4,6,8)*c(2,3,4)
[1] 4 12 24 16
Warning message:
In c(2, 4, 6, 8) * c(2, 3, 4) :
  longer object length is not a multiple of shorter object length
> c(2,4,6,8)/c(2,3,4)
[1] 1.0000000 1.3333333 1.5000000 4.0000000
Warning message:
In c(2, 4, 6, 8)/c(2, 3, 4) :
  longer object length is not a multiple of shorter object length
> |

```

So, let me try to first show you these things on the R console before I move forward. So, you can see here that I try to first clear the screen by pressing here Ctrl L. And now if I try to take here the say, any vector 1,2,3, and 4. So, this is c 1,2,3,4, you have to be very careful that c is going to be any small numerator that is lowercase and if you try to multiply it by here, suppose you have 5 you can see here the answer comes out to be 1 into 5 is 5, 2 into 5 is 10, 3 into 5 is 15, and 4 into 5 is 20.

And similarly, if you try to take 2 numbers here, c 5 comma 6 in the second data vector. So, once again this 5 and 6 are going to be multiplied with the first 2 positions 1 and 2. And then once again, this 5 and 6 are going to be multiplied with 3 and 4 respectively. So, you can see here it is not difficult. And similarly, if you try to take here the port numbers in the second data vector, suppose I take 5,6,7,8, in the second data vector, 4 elements and 4 elements in the first data vector 1,2,3,4, then you can see that this multiplication is happening element-wise. So, you can see here this 5,6,7,8 they have been multiplied at the respective positions in the first data vector. So, there is no issue.

And similarly, if you try to take here means another data vectors here 2,4,6, and 8, and you try to divide it by here 2, you can see here that this operation is coming out to be the similar that each and every element is going to be divided by the scalar 2. And similarly, if you try to divide it by suppose 2.4, then each of this element is going to be divided by 2.4. And now, similarly, if you try to take here in the division means 2 elements say 2 and here say 3. So, you can see here the answer is coming out to be like this, where 2 and 3, they are going to be

divided the number 2 and 4. And similarly, 2 and 4 are once again trying to divide the numbers 6 and 8. So, this is here the final outcome.

And if you try to click here, the same number of elements means in both the data vectors, then you can see here that this 2,3,4,5, in the second data vector, they will divide the number 2,4,6,8, in the first data vector and you will get this outcome. Now, in case if I try to take here an example, where I tried to first take the total number of elements in the two data vectors to be not the same. And I try to do here the multiplication you can see here that 2, 3, and 4, they are going to be multiplied with the 2, 4, and 6, but after that, this 8 will be multiplied by 2 and then there is no place where the 3 and 4 can be multiplied. So, it will give you here a warning message.

And similarly, if you try to do the same operation with the division sign also you will get here the similar outcome that this 2, 3, and 4, they are going to divide the numbers 2, 4, 6, which are the first 3 elements in the data vector 1 and this 8 is going to be divided by 2 and we get here and answer 4, but after this, there is no place where these 3 and 4 can operate. So, that is why it is trying to give you here a warning message. So, now, let us try to come back to our slides and we come to our last operations on the power operators.

(Refer Slide Time: 33:50)

Power Operators in Data Vectors:

`> c(2,3,4,5)^2`
`[1] 4 9 16 25`

command: application to a vector
output

$2^2, 3^2, 4^2, 5^2$

R Console

```
> c(2,3,4,5)^2
[1] 4 9 16 25
>
```

20

So, that is also pretty simple that if you try to take here power operator like `c(2,3,4,5)`, data vector and if you try to write down here had to then this power operator is going to operate on each of the numbers inside this data vector 1. So, this will become 2 square, 3 square, 4

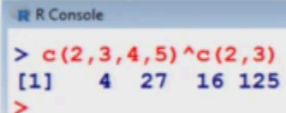
square, and 5 square. And you will get here an answer 4, 9, 16, and 25. And this is here the screenshot.

(Refer Slide Time: 34:18)

Power Operators in Data Vectors:

```
> c(2,3,4,5)^c(2,3) # !!ATTENTION! Observe the operation
[1] 4 27 16 125      # output
```

$2^2, 3^3, 4^2, 5^3$



R Console

```
> c(2,3,4,5)^c(2,3)
[1] 4 27 16 125
>
```

23

And similarly, if you try to take here the power operators with respect to that two data vector instead of a scalar. Then let me take here the first example, where the total number of elements in the first data vector are 2,3,4,5. That means there are four elements and the total number of elements in the second data vector are 2 and 3, it is only 2 in number but they are the exact multiple.

So, the outcome will be that these 2 and 3, they will be transported to the first data vector and this will become here 2 and 3 as a power operator on the first two elements and once again, this 2 and 3 will be transported and this 4 and 5 will be operated by 4 square and 5 cube. So, now, here is the answer 4, 27, 16, 125, that you know that how would this 5 cube is obtained? Well, so, you can see here that is pretty straightforward and pretty simple.

(Refer Slide Time: 35:11)

Power Operators in Data Vectors:

```
> c(6,5,4,3,2,1)^c(2,3,4) # command: application  
# output  
[1] 36 125 256 9 8 1
```

$6^2, 5^3, 4^4, 3^2, 2^3, 1^4$



```
R Console  
> c(6,5,4,3,2,1)^c(2,3,4)  
[1] 36 125 256 9 8 1  
>
```

22

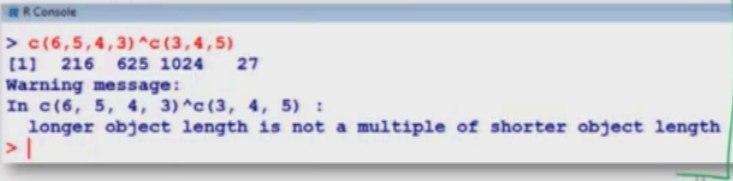
Now, similarly, if I try to take here one more example, where the total number of elements in the two data vectors are the exact multiple, so, in the first data vector, I have 6 numbers 6,5,4,3,2,1, and in the second data vector, I have 3 values 2,3,4. So, the total number of values vectors, they are just double of the total number of values in the second data vector. So, now, what will happen this 2,3,4, will be transported here and this will become here 2,3,4, and the power and once again, it will be transported to the first data vector and it will become here 2,3,4, in the power operator and then you will get here an outcome like this one 36, 125, 256, 9, 8, 1. And this is here the screenshot So, you can see here it is not a very difficult operation.

(Refer Slide Time: 36:02)

Power Operators in Data Vectors:

```
> c(6,5,4,3)^c(3,4,5) # Warning message  
[1] 216 625 1024 27 # output  
Warning message:  
In c(6,5,4,3)^c(3,4,5) : longer object length is  
not a multiple of shorter object length
```

$6^3, 5^4, 4^5, 3^3$



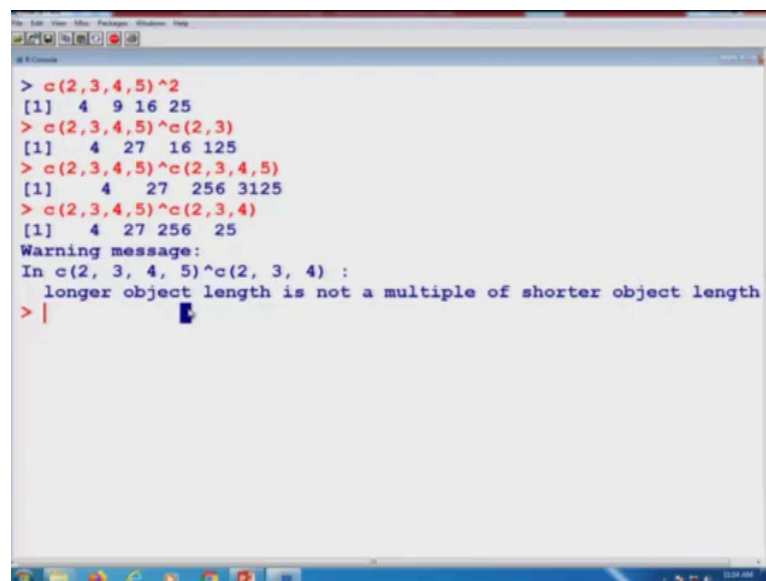
```
R Console  
> c(6,5,4,3)^c(3,4,5)  
[1] 216 625 1024 27  
Warning message:  
In c(6, 5, 4, 3)^c(3, 4, 5) :  
longer object length is not a multiple of shorter object length  
>
```

23

And if I try to take here the last example, where the total number of elements are not the exact multiple, then I considered here 4 elements in the data vector 1 and 3 elements in the second data vector. So, the 3,4,5, will be transported to the first data vector and it will be operated like 3,4,5, as a power operator on the first three elements and then it will become here in the last element 3 cube, but after that, there are no elements. So, there is no place where these power operators are 4 and 5 can be operated. So, R will not do anything, but it will give you here a message.

So, you get here this 6 cube is 216, 5 is two power 4 is 625 that is 5 into 5 is 25, 25 into 25 is 625. And then 4 raise power of 5 here is 1024 and 3 cube here is 27. But you are getting here warning, and this is here the screenshot of the same operation and we try to do it on the R console. So, you can see that it is not difficult, but let me try to show you these operations on the R console.

(Refer Slide Time: 37:05)



```

> c(2,3,4,5)^2
[1] 4 9 16 25
> c(2,3,4,5)^c(2,3)
[1] 4 27 16 125
> c(2,3,4,5)^c(2,3,4,5)
[1] 4 27 256 3125
> c(2,3,4,5)^c(2,3,4)
[1] 4 27 256 25
Warning message:
In c(2, 3, 4, 5)^c(2, 3, 4) :
  longer object length is not a multiple of shorter object length
> |

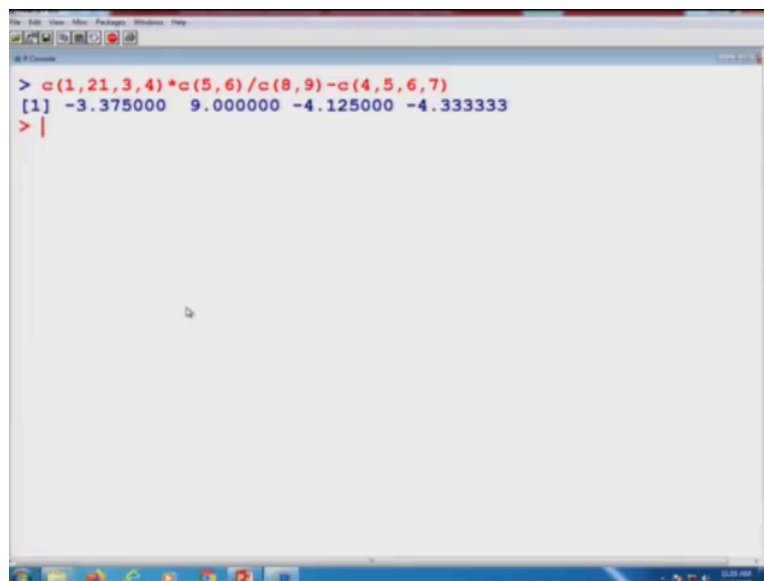
```

So, let me first try to clear the screen. And now, I try to take here some numbers 2,3,4, and 5 and I try to use the power operator and I want to square them. So that will become here 2 square which is 4, 3 square which is 9, 4 square which is 16, and 5 square which is 25. And then I try to take here means another example in which I tried to take the data vector instead of a scalar and the total number of elements in the second data vector are 2, which are 2 and 3 and they are the exact multiple of the total number of elements in the first data vector. So, if you try to operate it here, these 2 and 3 will be power operated, on this number 2 and 3 and then once again on 4 and 5.

So, this is an outcome of 2 square 3 cube, 4 square and 5 cube. And if you try to take here the same number of elements in the 2 data vector, so, this will become like this here that in the first data vector, we have element 2,3,4,5, and in the second data vector, we have the elements 2,3,4,5, once again. So, now, if you try to operate it again, this will be 2 square 3 cube, 4 raise to the power of 4, and 5 raised to the power of 5. So, this is your here the outcome. So, means all the power operations are getting distributed over the elements in the first data vector and if you try to reduce the total number of elements in the second data vector, so that they are not the exact multiple.

So, you can see here the similar operation will come that 2, 3, and 4 will be operated on 2, 3, and 4, but on 5 only 2 will be operated and after that, there is no place where 3 and 4 can operate. So, it will give you here a warning message. And now you see with this one thing I would like to show you here very clearly that I have used here these different mathematical operations on data vector 1 by 1.

(Refer Slide Time: 39:04)

A screenshot of an R console window. The command entered is `> c(1,21,3,4)*c(5,6)/c(8,9)-c(4,5,6,7)`. The output is `[1] -3.375000 9.000000 -4.125000 -4.333333`. The prompt `> |` is visible on the next line.

```
> c(1,21,3,4)*c(5,6)/c(8,9)-c(4,5,6,7)
[1] -3.375000 9.000000 -4.125000 -4.333333
> |
```

But here the BODMAS rule is also working. For example, if I try to say here, see, this first element having 1, 21, 3, and 4 values, which are multiplied by say here another element 5 and 6, and they are divided by say here another number here 8 and 9 and then they are affected by some number here 4,5,6,7. So, you can see here this operation will be happening following the same BODMAS rule.

So, you can see here that these operations are not a difficult, and well in case if you also want to use some brackets over here, I mean the same automatic rules are going to be applied over

here. Sorry for this long lecture. But these are very basic operations and you will see that we are going to use them very frequently. So, it was very important for me to just cover up all the things in a single lecture so that you can continue with it. Well, you have an option to view this lecture into 2 parts.

So, now I will stop in this lecture and I hope that it is not a very difficult lecture for you to understand. These are the same operations that you had learned earlier in the R courses but If you have not learned it at least I have given you a fair idea, that how the R operations are working. So, if you have not learned it earlier, I will say just take some more examples and try to execute by yourself. Just try to take only two elements or three elements and try to operate manually and try to see the same operation after doing it in the R console and see are you getting the same answer? I am sure you will get the same answer and If not then you have to see what is really happening between your thought process and the thought process of the R software.

So, I will request you all just take some examples and try to practice it. And I will see you in the next lecture with more topics on the R software. Till then, goodbye.